```c
int *px = &x; // &x = Adresse des ints, * = Pointer-Bezeichner
int y = *px;  // *px = Wert einer int-Adresse, y = 5, * = Dereferenzoperator
```

1 (16^0), 16 (16^1), 256 (16^2), 4'096 (16^3), 65'536 (16^4), 1'048'576 (16^5)

BETRIEBSSYSTEM API - ZUSAMMENSPIEL OS UND PROGRAMM

Ein Programm kann über Systemaufrufe auf Dienste des OS zugreifen: syscall → Kernel Mode des Prozessors → Argumente Registern übergeben → Argument für Nummer des Systemaufrufs
OS Mechanismen um Programme zu konfigurieren: Programmargumente (explizit), Umgebungsv. (implizit), Dateien oder Windows Registry (KVP).
Aufgaben: Abstraktion, Portabilität, Ressourcenmanagement & Isolation der Anwendungen, Benutzerverwaltung und Sicherheit
Privilege Levels: **Kernel-Mode** *(darf alles ausführen, Ring 0)*, **User-Mode** *(darf nur beschränkte Menge an Instruktionen ausführen, Ring 3)*
Kernels: **Microkernel** *(nur kritische Teile laufen im Kernel-Mode)*, **Monolithisch** *(meiste OS, weniger Wechsel, weniger Schutz)*, **Unikernel** *(Kernel ist nur ein Programm)*
`syscall`: Befehl um auf OS-Dienste zuzugreifen, wechselt in Kernel Mode von Prozessor. Argumente in Register gemäss OS *(Ein Argument ist die Syscall Nummer/Code pro OS-Kernel-Funktion. z.B. exit=60, exec*.)*. C-API sollte nicht Syscall aufrufen, sondern C-Wrapper-Funktionen.
ABI: Application Binary Interface, Abstrakte Schnittstelle mit platformunabhängigen Aspekten
API: Application Programming Interface, Konkrete Schnittstellen, Calling Convention, Abbildung von Datenstrukturen. **Linux-Kernels** sind API-, aber nicht ABI-kompatibel. *(C-Wrapper-Funktionen)*
POSIX: Portable Operating System Interface. Sammlung von IEEE Standards, welche die Kompatibilität zwischen OS gewährleistet. Windows ist nicht POSIX-konform.

PROGRAMMARGUMENTE

`clang -c abc.c`. Shell teilt Programmargumente in Strings auf *(Trennung durch Leerzeichen, sonst Quotes)*. Programmarg. explizit angeben, nützlich für Informationen, die bei jedem Aufruf anders sind.

Calling Convention

- OS schreibt Argumente als null-terminierte Strings in den Speicherbereich des Programms.
- Zusätzlich legt das OS ein Array `argv` an, dessen Elemente jeweils auf das erste Zeichen eines Arguments zeigen.
- Der Pointer auf dieses Array und die Anzahl der Elemente wird dem Progamm an einer vom OS definierten Stelle zur Verfügung gestellt, z.B. in Registern oder auf dem Stack.
- `int main(int argc, char * argv[]) { ... } // argv[0] = program path`

UMGEBUNGSVARIABLEN

Umgebungsv.: Strings, die mindestens ein `Key=Value` enthalten *OPTER=1, PATH=/home/ost/bin*. Unter POSIX verwaltet das OS die Umgebungsvariablen innerhalb jedes laufenden Prozesses. Werden initial festgelegt. Das OS legt die Variablen als ein null-terminiertes Array von Pointern auf null-terminierte Strings ab.
Unter C zeigt die Variable `extern char **environ` darauf.

`char *value = getenv("PATH")`: Abfragen einer Umgebungsv. *returns Adresse von 1. Zeichen oder 0 wenn nicht vorhanden*
`int ret = setenv("HOME", "/usr/home", 1) / setenv(*key, *value, overwrite)`: Umgebungsv. setzen *Mit oder ohne überschreiben*
`int ret = putenv("HOME=/usr/home") / putenv(char * string)`: *löscht vorhandenen key, string als Variable mitgeben ist im nachhinein modifizierbar (gefährlich Pointer auf NULL)*
`int ret = unsetenv("HOME")`: Umgebungsv. entfernen

DATEISYSTEM API

Datei: Daten (Bytes) und Metadaten (Sammlung von Attributen) *Dateiname, Verzeichnis, Ablageort*
Verzeichnis: Datei mit Liste aller Dateien. Verz.Hierarchie = Gesamtheit aller Verz. im System
Links: **Hard-Link**: gleicher Inode, verschiedene Pfade, **Symbolischer Link**: wie eine Datei die Pfad auf andere Datei enthält. 0-1 Block alloziert.
Zugriffsrechte: Je 3 Permission-Bits für Owner, Gruppe und andere Benutzer.
`r=4, w=2, x=1` *Zahlen so addieren, Beispiel: 0741 = rwx r-- --x*
Arbeitsverzeichnis: Bezugspunkt für relative Pfade, jeder Prozess hat eines *(getcwd(), chdir(): nimmt String, fchdir(): nimmt File Deskriptor)*. z.B. `/home/bsys2`
Pfade: Absolut *(beginnt mit /)*, Relativ *(beginnt nicht mit /)*, Kanonisch *(Absolut, ohne «.» und «..», realpath())* Längster erlaubter Pfadname `NAME_MAX` *Maximale Länge eines Dateinamens (exklusive terminierender Null)* je nach System unterschiedlich, aber gemäss `_POSIX_NAME_MAX` mindestens 14 *limits.h*
Datentyp: Applikationen müssen Daten validieren und auf Grenzverletzung überprüfen.
POSIX-API: für direkten Zugriff, alle Dateien sind rohe Binärdaten. Nutzt File-Deskriptoren. Blockorientiert, liest Bytes. `open/close, read/write, lseek, pwrite/pread`
C-API: für direkten Zugriff auf Streams, Stream-orientiert (möglicherweise gepuffert) liest characters (Zeilenumbruch automatisch auf OS formatiert), OS-unabhängig. `fopen/fclose, fgetc/fputc/funputc, fseek/ftell`
File-Deskriptor: Index in die File-Descriptor-Table vom aktuellen Prozess.
File-Descriptor-Table (FDT): hat jeder Prozess, Einträge enthalten u.a. aktuellen Offset + Zeiger in Global File Table. Nutze Indizes in diese Tabelle als Argument für Filesystem-API-Funktionen.
stdin, stdout, stderr: vordefinierte Streams in jedem FDT von 0-1, wenn geschlossen können diese nicht mehr einfach geöffnet werden (im Prozess).
`errno`: Makro oder globale Variable vom typ `int`. Nach Auftreten eines Fehlers abfragen.

FILE-DESCRIPTOR (FD)

```c
#define N 32
int main (int argc, char * argv[]) {
    // Gibt Arbeitsverzeichnis aus
    char *wd = malloc(PATH_MAX);
    getcwd(wd, PATH_MAX); printf("Current WD is %s", wd);
    free(wd);

    // Lesen und Schreiben von Dateien
    char buf[N]; // oben definiert
    char spath[PATH_MAX]; /* source path */
    char dpath[PATH_MAX]; /* destination path */
    // ... get paths from somewhere
    int src = open(spath, O_RDONLY);
    int dst = open(dpath, O_WRONLY | O_CREAT, S_IRWXU);
    ssize_t read_bytes = read(src, buf, N);
    write(dst, buf, read_bytes);// if file closed early, only write as many bytes
    close(src);                 // as have been read from the file
    close(dst);

    // Datei öffnen zum lesen und Fehlerbehandlung
    int fd = open("myfile.dat", O_RDONLY);
    if (fd < 0) {
        // textuelle Beschreibung vom Fehlercode:
        printf ("Error: %s\n", strerror (errno));
        if (errno == EACCESS) { printf("access denied"); } // eigener Text
        perror ("chdir"); // (add error) print to standard error stream
        // error stream neu "chdir:`strerror(errno)`"
    }
    /* read data; */ close(fd);
} // main
```

Files werden in der POSIX-API über FD's repräsentiert. Returnt **Index in Tabelle** aller geöffneter Dateien im Prozess → Enthält **Index in systemweite Tabelle** → Enthält Daten zur Identifikation der Datei. `STDIN_FILENO = 0`: *standard input*, `STDOUT_FILENO = 1`: *standard output*, `STDERR_FILENO = 2`: *standard error*

`int open (char *path, int flags, ...)`: erzeugt FD auf Datei an `path`. `flags` = wie die Datei geöffnet werden soll: `O_RDONLY`, `O_RDWR`, `O_CREAT`(wenn nicht existiert), `O_APPEND` (Setze Offset ans Ende der Datei vor jedem Schreibzugriff), `O_TRUNC` (Setze Länge der Datei auf 0)
`int close (int fd)`: schliesst Datei = dealloziert den FD. FD kann von `open` für andere Dateien verwendet werden. Returned 0 oder −1. *Wenn mehrere FDs die gleiche Datei öffnen, können sie sich gegenseitig Daten überschreiben.*
`ssize_t read(int fd, void * buffer, size_t n)`: Versucht die nächsten n Bytes am aktuellen Offset von fd in den Buffer zu kopieren, erhöht FD offset, blockierend. returned: Anzahl gel. Bytes.
`ssize_t write(int fd, void * buffer, size_t n)`: Versucht die nächsten n Byte vom buffer an aktuellen Offset von fd zu kopieren, erhöht FD offset, blockierend
`off_t lseek(int fd, off_t offset, int origin)`: Setzt offset auf `offset` und gibt neuen Offset zurück. `origin` = wozu Offset relativ ist: `SEEK_SET`: Beginn der Datei, `SEEK_CUR`: Aktueller Offset, `SEEK_END`: Dateieende.
Beispiele: `lseek(fd, 0, SEEK_CUR)` aktueller Offset, `lseek(fd, 0, SEEK_END)`: Grösse der Datei. `lseek (fd, n, SEEK_END)` hängt, bei nachfolgendem write, n Nullen an Datei.
`ssize_t pread/pwrite(int fd, void * buffer, size_t n, off_t offset)`: Wie `read/write`. Statt Offsets von `fd` wird zusätzlicher Parameter `offset` verwendet. fd offset nicht verändert.

Unterschiede Windows und POSIX

In Windows werden Pfad-Bestandteile durch Backslash (\) getrennt + es gibt ein Wurzelverzeichnis pro Datenträger/Partition + andere File-Handling-Funktionen. (z.B. open CreateFile)

C STREAM API (FILE API)

Unabhängig vom Betriebssystem (POSIX/Windows), Stream-basiert = zeichen-orientiert, für Dateien im Normalfall gepuffert, hat einen eigenen File-Position-Indicator (FPI).
FPI: bei gepuffert Position im Puffer, bei ungepuffert Offset des FD.
Streams: `FILE *` enthält **Informationen über einen Stream**. Soll nicht direkt verwendet oder kopiert werden, sondern nur über von C-API erzeugte Pointer.
FILE: Stream-infos, nur über C-API nutzen und nicht kopieren, die 3 Standard streams stdin-stderr.
`FILE * fopen(char const *path, char const *mode)`: **Öffnen eine Datei.** Erzeugt `FILE`-Objekt für Datei an `path`. Flags für `mode`: **r**, **w** in neue/bestehende geleerte Datei schreiben, **a** in neue/bestehende Datei anfügen, **r+** bestehende Datei lesen und schreiben, **w+** neue/geleerte bestehende Datei lesen und schreiben, **a+** neue/bestehende Datei lesen und anfügen
Gibt Pointer auf erzeugtes `FILE`-Objekt zurück oder 0 bei Fehler.
`FILE * fdopen(int fd, char const * mode)`: ist gleich, aber statt Pfad wird direkt der FD übergeben.
`int fileno (FILE *stream)`: gibt FD zurück von Stream (nicht mit POSIX mischen).
`int fclose(FILE *file)`: **Schliesst eine Datei.** Ruft `fflush()` *(schreibt Inhalt aus Speicher in die Datei)* auf, schliesst den Stream, entfernt `file` aus Speicher und gibt 0 zurück wenn OK, andernfalls `EOF`.
`int fgetc(FILE *stream)`: **Liest** das nächste Byte und erhöht FPI um 1.
`char * fgets(char *buf, int n, FILE *stream)`: liest bis zu $n-1$ Zeichen aus `stream`.
`int ungetc(int c, FILE *stream)`: **Lesen rückgängig machen.** Nutzt Unget-Stack.
`int fputc(int c, FILE *stream)`: schreibt `int c` als unsigned char in Stream.
`int fputs(char *s, FILE *stream)`: schreibt die Zeichen vom String `s` bis zur terminierenden 0 in `stream`.

Dateiende und Fehler:
`int feof(FILE *stream)` gibt 0 zurück, wenn Dateiende **noch nicht** erreicht wurde
`int ferror(FILE * stream)` gibt 0 zurück, wenn **kein** Fehler auftrat.

Manipulation des File-Position-Indicator (FPI):
`long ftell(FILE *stream)` gibt den gegenwärtigen FPI zurück, `int fseek (FILE *stream, long offset, int origin)` setzt den FPI, analog zu `lseek`, `int rewind (FILE *stream)` setzt den Stream zurück.

PROZESSE

Modell Monoprogrammierung: Prozess sieht System, als gäbe es nur ihn und OS.
Programm: Ein Programm kann mehrfach ausgeführt werden: verschiedene, voneinander unabhängige Prozesse. Prozess kann mehrere Programme nacheinander ausführen.
Prozesse: sind die **Verwaltungseinheit** des OS für Programme. Jedem Prozess ist ein **virtueller Adressraum** zugeordnet (MMU). **Ein Prozess umfasst**: das **Abbild eines Programms** im Hauptspeicher *(text section)*, die **globalen Variablen des Programms** *(data section)*, Speicher für **Heap** und Speicher für **Stack**.
Process Control Block (PCB): IDs, Kontext, Scheduling-Infos *Priorität*, Sync-Daten, offene Dateien...
Kontext: Register, Flags, Instruction Pointer, MMU-Konfiguration (Page-Table-Pointer)
Kontextwechsel: Interrupt → context save *von Prozess in PCB* → **Interrupt-Handler** *kann Kontext überschreiben* → context restore. Ermöglicht quasi-parallele Ausführung.
Prozess-Erstellung: OS erzeugt Prozess und lädt das Programm in den Prozess. `fork exec`
Prozess-Hierarchie: Baumstruktur, startet bei Prozess 1.

PROZESS-API

`pid_t fork(void)` erzeugt **exakte Kopie** (C) als Kind des Prozesses (P), mit eigener Prozess-ID (>0). Die Funktion führt in **beiden** Prozessen den Code an derselben Stelle fort.
`void exit(int code)`: Beendet das Programm und gibt `code` zurück.
`pid_t wait(int *status)`: unterbricht Prozess, bis Child beendet wurde.
`pid_t waitpid (pid_t pid, int *status, int options)`: wie `wait()`, aber `pid` bestimmt, auf welchen Child-Prozess man warten will *(> 0 = Prozess mit dieser ID, −1 = irgendeinen, 0 = alle C mit der gleichen Prozessgruppen-ID)*.

```c
void spawn_worker (...) {
  if (fork() == 0) { /* do something in worker process; */ exit(0); }
}
for (int i = 0; i < n; ++i) { spawn_worker(...); }
// ... do something in parent process
do { pid = wait(0); } while (pid > 0 || errno != ECHILD); //wait for all children
```

`exec()`-Funktionen: Jede davon **ersetzt** im gerade laufenden Prozess das Programmimage **durch ein anderes**. Programmargumente müssen spezifiziert werden. *(..l als Liste, ..v als Array)*

Aufrufen der Executable	Zustand der Environment-Variabeln	Programmargumente als Liste	Programmargumente als Array
Angabe des Pfads	mit neuem Environment	`execle()`	`execve()`
	mit altem Environment	`execl()`	`execv()`
Suche über `PATH`		`execlp()`	`execvp()`

Zombie- & Orphan-Prozesse

Dauerhafter Zombie-Prozess: P ruft wegen Fehler `wait()` nie auf. **Orphan-Prozess:** P wird vor C beendet. P kann somit nicht mehr auf C warten, was bei Beendung von C in einem dauerhaften Zombie resultiert. Wenn P beendet wird, werden deshalb alle C an Prozess mit `pid=1` übertragen, der `wait()` in einer Endlosschleife aufruft.

`unsigned int sleep (unsigned int seconds)`: **unterbricht** Ausführung, bis eine Anzahl Sekunden ungefähr verstrichen ist. Gibt vom Schlaf noch vorhandene Sekunden zurück.
`int atexit (void (*function)(void))`: Registriert Funktionen für Aufräumarbeiten vor Ende.
`pid_t getpid()/getppid()` geben die (Parent-)Prozess-ID zurück.

PROGRAMME UND BIBLIOTHEKEN

`"C-Quelle" → Präprozessor → "Bereinigte C-Quelle" → Compiler → "Assembler-Datei" → Assembler → "Objekt-Datei" → Linker → "Executable"`
Präprozessor: Die Ausgabe des Präprozessors ist eine reine C-Datei (Translation-Unit) ohne Makros, Kommentare oder Präprozessor-Direktiven. **Linker:** Der Linker verknüpft Objekt-Dateien (und statische Bibliotheken) zu Executables oder dynamischen Bibliotheken. **Loader** lädt Executables und eventuelle dynamische Bibliotheken dieser in den Hauptspeicher.

ELF (EXECUTABLE AND LINKING FORMAT)

Binär-Format, das Kompilate spezifiziert. Besteht aus ***Linking View*** *(wichtig für Linker, für Object-Files und Shared Objects, Sektionen)* und ***Execution View*** *(wichtig für Loader, für Programme und Shared Objects, Segmente)*. Die Views definieren «gleichartige» Daten. Der **Linker** vermittelt zwischen beiden Views.
ELF Struktur: Besteht aus **Header**, ***Programm Header Table***, ***Segmenten***, ***Section Header Table***, ***Sektionen*** und enthält auch Strings (z.B. Namen von Symbolen + Sektionen..) und Symboltabelle (für Bibliotheken und object files).

Header: Beschreibt den **Aufbau** der Datei: Typ, 32/64-bit, Encoding, Maschinenarchitektur, Entrypoint, Infos zu den Einträgen in PHT und SHT.
Segment/Program Header Table und Segmente: Tabelle mit n Einträgen, jeder Eintrag *(je 32 Byte)* beschreibt ein Segment *(Typ und Flags, Offset und Grösse, virtuelle Adresse und Grösse im Speicher - kann unterschiedlich zur Dateigrösse sein)*. Ist Verbindung zwischen Segmenten im RAM und im File. Definiert, wo ein Segment liegt und wohin der Loader es im RAM laden soll.
Segmente werden vom **Loader** dynamisch **zur Laufzeit** verwendet.
Section Header Table und Sektionen: Tabelle mit m Einträgen ($\neq n$). Jeder Eintrag *(je 40 Byte)* beschreibt eine Sektion *(Name, Section-Typ, Flags, Offset und Grösse, ...)*. Werden vom **Linker** verwendet: Verschmilzt Sektionen und erzeugt ausführbares Executable.
String-Tabelle: Bereich in der Datei, der nacheinander **null-terminierte Strings enthält**. Strings werden relativ zum Beginn der Tabelle referenziert. enthält z.B. Namen von Symbolen. Keine «String-Literale», die sind in .rodata.
Symbole & Symboltabelle: Die Symboltabelle enthält jeweils **einen Eintrag je Symbol** *(16 Byte: 4B Name, 4B Wert, 4B Grösse, 4B Info)*.

BIBLIOTHEKEN

Statische Bibliotheken: Archive von Objekt-Dateien. Name: `lib<name>.a`, referenziert wird nur `<name>`. **Linker** behandelt statische Bibliotheken wie **mehrere Objekt-Dateien**. Ursprünglich gab es **nur statische Bibliotheken** *(Einfach zu implementieren, aber Funktionalität fix, wie zip vorstellen)*.
Dynamische Bibliotheken: Linken zur Ladezeit bzw. Laufzeit des Programms. Executable enthält nur Referenz auf Bibliothek. Höherer Aufwand. **Vorteile**: austauschbar, Entkoppelter Lebenszyklus, Plugin-Mechanismus, (Schnellere Ladezeiten durch Lazy Loading, Flexibler Funktionsumfang). **Dynamisches Linken** explizit im Programm *dlopen, dlsym* oder zum Programmstart.

POSIX SHARED OBJECTS API

Konventionen: Shared Objects können **automatisch** bei Bedarf geladen werden. Linker→Linker-Namen, Loader→SO-Namen.
- **Linker-Name:** `lib + Bibliotheksname + .so` *(z.B. libmylib.so)*,
- **SO-Name:** `Linker-Name + . + Versionsnummer` *(z.B. libmylib.so.2)*,
- **Real-Name:** `SO-name + . + Unterversionsnummer` *(z.B. libmylib.so.2.1)*

Shared Objects: Nahezu alle Executables benötigen **zwei Shared Objects**: `libc.so`: Standard C library, `ld-linux.so`: ELF Shared Object loader *(Lädt Shared Objects und rekursiv alle Dependencies)*.
Implementierung dynamischer Bibliotheken: Müssen verschiebbar sein, mehrere müssen in den gleichen Prozess geladen werden. Die Aufgabe des Linkers wird in den Loader bzw. Dynamic Linker verschoben *(Load Time Relocation)*.

`void * dlopen (char * filename, int mode)`: **öffnet** eine dynamische Bibliothek und gibt ein Handle darauf zurück. `mode` ist einer der folgenden Werte:
- `RTLD_NOW`: Alle Symbole werden beim Laden gebunden
- `RTLD_LAZY`: Symbole werden bei Bedarf gebunden
- `RTLD_GLOBAL`: Symbole können beim Binden anderer Objektdateien verwendet werden
- `RTLD_LOCAL`: Symbole werden nicht für andere Objektdateien verwendet

`void * dlsym (void * handle, char * name)`: gibt die **Adresse des Symbols** `name` aus der mit `handle` bezeichneten **Bibliothek** zurück. Keine Typinformationen *(Variabel? Funktion?)*

```c
// type "func_t" is an address of a function with a int param and int return type
typedef int (*func_t)(int);
handle = dlopen("libmylib.so", RTLD_NOW); // open library
func_t f = dlsym(handle, "my_function"); // write my_function addr. into a func_t
int *i = dlsym(handle, "my_int"); // get address of "my_int"
(*f)(*i); // call "my_function" with "my_int" as parameter
```

`int dlclose (void * handle)`: schliesst das durch `handle` bezeichnete, zuvor geöffnete Objekt.
`char * dlerror()`: gibt Fehlermeldung als null-terminierten String zurück.

Position-Independent Code: *(Adressen nur relativ zum Instruction Pointer, Prozessor muss relative Instruktionen anbieten)* Dynamische Bibliotheken sollen **Code zwischen Programmen teilen**. Code soll **nicht mehrfach** im Speicher abgelegt werden.
Mit Shared Memory: kann jedes Programm eine **eigene virtuelle Page** für den Code definieren. Diese werden auf denselben Frame im RAM gemappt.
Relative Moves via Relative Calls: Mittels Hilfsfunktion wird Rücksprungadresse in Register abgelegt, somit kann relativ dazu gearbeitet werden.
Global Offset Table (GOT): Pro dynamische Bibliothek & Executable vorhanden, enthält pro Symbol einen Eintrag. Der Loader füllt zur Laufzeit die Adressen in die GOT ein.
Procedure Linkage Table (PLT): Implementiert Lazy Binding. Enthält pro Funktion einen Eintrag, dieser enthält Sprungbefehl an Adresse in GOT-Eintrag. Dieser zeigt auf eine Proxy-Funktion, welche den GOT-Eintrag überschreibt. **Vorteil:** erspart bedingten Sprung.

THREADS

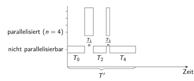
Jeder **Prozess** hat virtuell den **ganzen Rechner** für **sich alleine**. **Prozesse** sind gut geeignet für **unabhängige Applikationen**. Nachteile: Realisierung **paralleler Abläufe** innerhalb derselben Applikation ist **aufwändig**. **Overhead** zu gross falls nur kürzere Teilaktivitäten, **gemeinsame Ressourcennutzung** ist **erschwert**.

Threads: parallel ablaufende Aktivitäten innerhalb eines Prozesses, welche auf **alle** Ressourcen im Prozess gleichermassen Zugriff haben. Benötigen **eigenen Kontext** und **eigenen Stack**. Informationen werden in einem **Thread-Control-Block** abgelegt.

AMDAHLS REGEL

Nur bestimmte Teile eines Algorithmus können parallelisiert werden.
T: Ausführungszeit, wenn **komplett seriell** durchgeführt *(Im Bild: $T = T_0 + T_1 + T_2 + T_3 + T_4$)*
n: Anzahl der Prozessoren
T': Ausführungszeit, wenn **maximal parallelisiert** *gesuchte Grösse*
T_s: Ausführungszeit für den Anteil, der **seriell** ausgeführt werden **muss** *(Im Bild: $T_s = T_0 + T_2 + T_4$)*
$T - T_s$: Ausführungszeit für den Anteil, der **parallel** ausgeführt werden **kann** *(Im Bild: $T_1 + T_3$)*

$(T - T_s)/n$: Parallel-Anteil verteilt auf alle n Prozessoren. *(Im Bild: $(T_1 + T_3)/n$)*
$T_s + \frac{T-T_s}{n}$: Serieller Teil + Paralleler Teil $= T'$



Die **serielle Variante** benötigt also höchstens **f mal mehr Zeit** als die **parallele Variante**:

$$f \le \frac{T}{T'} = \frac{T}{T_s + \frac{T-T_s}{n}}$$

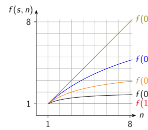
f heisst auch **Speedup-Faktor**, weil die parallele Variante max. f-mal schneller ist als die serielle.

Definiert man $s = T_s/T$, also den seriellen Anteil am Algorithmus, dann ist $s \cdot T = T_s$. Dadurch erhält man f unabhängig von der Zeit:

$$f \le \frac{T}{T_s + \frac{T-T_s}{n}} = \frac{T}{s\cdot T + \frac{T-s\cdot T}{n}} = \frac{T}{s\cdot T + \frac{1-s}{n}\cdot T} \quad\Rightarrow\quad f \le \frac{1}{s + \frac{1-s}{n}}$$

Bringe s und $\cdot\frac{\cdot}{n}$ auf den gleichen Nenner

- Amd. Regel = Abschätzung einer **oberen Schranke** für den maximalen Geschwindigkeitsgewinn
- Nur wenn **alles** parallelisierbar, ist Speedup **proportional** und **maximal** *100% parallel: $f(0,n) = n$ und rein seriell: $f(1,n)$*
- Sonst ist Speedup mit **höherer Prozessor-Anzahl** immer **geringer** *(Kurve flacht ab)*



Grenzwert: Mit höherer Anzahl Prozessoren nähert sich der Speedup $\frac{1}{s}$ an:

$$\lim_{n\to\infty} \frac{1-s}{n} = 0 \qquad \lim_{n\to\infty} s + \frac{1-s}{n} = s \qquad \lim_{n\to\infty} \frac{1}{s + \frac{1-s}{n}} = \frac{1}{s}$$

POSIX THREAD API

```c
int pthread_create(
    pthread_t * thread_id, pthread_attr_t const *attributes,
    void * (*start_function) (void *), void *argument)
```

erzeugt einen Thread, die ID des neuen Threads wird im Out-Parameter `thread_id` zurückgegeben. `attributes` ist ein opakes Objekt, mit dem z.B. die Stack-Grösse spezifiziert werden kann. Die erste auszuführende Instruktion ist die Funktion in `start_function`. `argument` ist ein Pointer auf eine Datenstruktur auf dem Heap für die Argumente für `start_function`.

```c
// Erstellung
struct T { // params of function
  int value;
};
void * my_start (void * arg) {
  struct T * p = arg;
  printf ("%d\n", p->value);
  free (arg);
  return 0:
}
```

```c
// Verwendung
void start_my_thread (void) {
  struct T * t = malloc (sizeof (struct T));
  t->value = 109; // set argument
  pthread_t tid;
  pthread_create (
    &tid,
    0, // default attributes
    &my_start,
    t);}
```

Thread-Attribute:

```c
pthread_attr_t attr; // Variabel erstellen
pthread_attr_init (&attr); // Variabel initialisieren
pthread_attr_setstacksize (&attr, 1 << 16); // 64kb Stackgrösse
pthread_create (..., &attr, ...); // Thread erstellen
pthread_attr_destroy (&attr); // Attribute löschen
```

Lebensdauer: Lebt solange, bis er aus der Funktion `start_function` zurückspringt, er `pthread_exit` oder ein anderer Thread `pthread_cancel` aufruft oder sein Prozess beendet wird.

`void pthread_exit (void *return_value)`: **Beendet** den Thread und gibt den `return_value` zurück. Das ist äquivalent zum Rücksprung aus `start_function` mit dem Rückgabewert.
`int pthread_cancel (pthread_t thread_id)`: Sendet eine **Anforderung**, dass der Thread mit `thread_id` **beendet** werden soll. Die Funktion **wartet nicht**, dass der Thread **tatsächlich beendet** wurde. Der Rückgabewert ist 0, wenn der Thread existiert, bzw. `ESRCH` *(error_search)*, wenn nicht.
`int pthread_detach (pthread_t thread_id)`: **Entfernt den Speicher**, den ein Thread belegt hat, falls dieser **bereits beendet** wurde. Beendet den Thread aber **nicht**. *(Erstellt Daemon Thread)*
`int pthread_join (pthread_t thread_id, void **return_value)`: **Wartet** solange, bis der Thread mit `thread_id` **beendet** wurde. Nimmt den **Rückgabewert** des Threads im Out-Parameter `return_value` entgegen. Dieser kann `NULL` sein, wenn nicht gewünscht. Ruft `pthread_detach` auf.
`pthread_t pthread_self (void)`: Gibt die **ID** des **gerade laufenden** Threads zurück.

THREAD-LOCAL STORAGE (TLS)

TLS ist ein Mechanismus, der **globale Variablen per Thread** zur Verfügung stellt. Dies benötigt mehrere explizite Einzelschritte: **Bevor Threads erzeugt werden:** Anlegen eines **Keys**, der die TLS-Variable **identifiziert**, **Speichern** des Keys in einer **globalen Variable** **Im Thread:** **Auslesen** des Keys aus der globalen Variable, **Auslesen / Schreiben** des Werts anhand des Keys.

`int pthread_key_create(pthread_key_t *key, void (*destructor) (void*))`: Erzeugt einen **neuen Key** im Out-Parameter `key`. **Opake Datenstruktur**. Am Thread-Ende Call auf `destructor`.
`int pthread_key_delete(pthread_key_t key)`: **Entfernt den Key** und die entsprechenden Values aus allen Threads. Der Key darf nach diesem Aufruf **nicht mehr verwendet** werden. Sollte erst aufgerufen werden, wenn alle dazugehörende Threads beendet sind.
`int pthread_setspecific(pthread_key_t key, const void * value)`
`void * pthread_getspecific(pthread_key_t key)` **schreibt** bzw. **liest** den Wert, der mit dem Key in diesem Thread assoziiert ist. Oft als **Pointer auf einen Speicherbereich** verwendet.

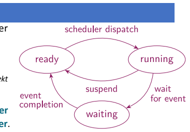
```c
// Setup
typedef struct {
  int code;
  char *message;
} error_t;
pthread_key_t error;
void set_up_error (void) {
  // am Anfang des Threads aufgerufen
  pthread_setspecific(
   error, malloc(sizeof(error_t)))}
```

```c
// Lesen und Schreiben im Thread
void print_error (void) {
  error_t * e =
pthread_getspecific(error);
  printf("Error %d: %s\n",
    e->code, e->message);}
int force_error (void) {
  error_t * e =
pthread_getspecific(error);
  e->code = 98;
  e->message = "file not found";
  return -1;}
```

```c
// Main und Thread
void *thread_function (void *) {
  set_up_error();
  if (force_error () == -1) { print_error (); }
}
int main (int argc, char * argv[]) {
  pthread_key_create (&error, NULL); // Key erzeugen
  pthread_t tid;
  pthread_create (&tid, NULL, &thread_function, NULL); // Threads erzeugen
  pthread_join (tid, NULL);
}
```

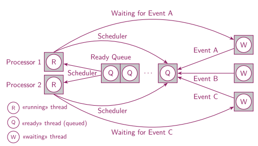
SCHEDULING

Auf einem Prozessor läuft zu einem Zeitpunkt immer **höchstens ein Thread**. Es gibt folgende Zustände:
- **Running** *(der Thread, der gerade läuft)*
- **Ready** *(Threads die laufen können, es aber gerade nicht tun)*
- **Waiting:** *(Threads, die auf ein Ereignis warten, können nicht direkt in den Running State wechseln)*



Übergänge von einem Status zum anderen werden **immer vom OS** vorgenommen. Dieser Teil vom OS heisst **Scheduler**.

Das OS **registriert Threads** auf ein Ereignis und setzt sie in den Zustand «waiting». Tritt das Ereignis auf, ändert das OS den Zustand auf **ready**. *(Es laufen nur Threads auf dem Prozessor, die nicht warten.)*
Ready-Queue: In der Ready-Queue *(kann auch ein Tree sein)* befinden sich alle Threads, die **bereit sind zu laufen**.
Powerdown-Modus: Wenn kein Thread **laufbereit** ist, schaltet das OS den Prozessor in **Standby** und wird durch **Interrupt** wieder geweckt.



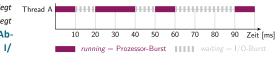
Arten von Threads: I/O-lastig *(Wenig rechnen, viel I/O-Geräte-Kommunikation)*, **Prozessor-lastig** *(Viel rechnen, wenig Kommunikation)*

Arten der Nebenläufigkeit

Kooperativ *(Threads entscheiden selbst über Abgabe des Prozessors)*
Präemptiv *(Scheduler entscheidet, wann einem Thread der Prozessor entzogen wird)*

Präemptives Multithreading: Thread läuft, bis er auf etwas zu **warten** beginnt, Prozessor **yielded**, ein **System-Timer-Interrupt** auftritt oder ein **bevorzugter Thread** erzeugt oder ready wird.
Parallele, quasiparallele und nebenläufige Ausführung: Parallel *(Tatsächliche Gleichzeitigkeit, n Prozessoren für n Threads)*, **Quasiparallel** *(n Threads auf $< n$ Prozessoren abwechselnd)*, **Nebenläufig** *(Überbegriff für parallel oder quasiparallel)*

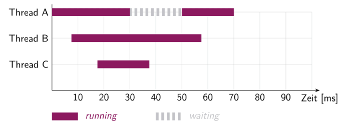
Bursts: Prozessor-Burst *(Thread belegt Prozessor voll)*, **I/O-Burst** *(Thread belegt Prozessor nicht)*. Jeder Thread = **Abfolge** von **Prozessor-Bursts** und **I/O-Bursts**.



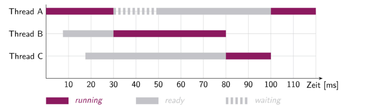
SCHEDULING-STRATEGIEN

Anforderungen Geschlossene Systeme *(Hersteller kennt Anwendungen und ihre Beziehungen)* vs. **Anford. Offene Systeme** *(Hersteller muss von typischen Anwendungen ausgehen)* **Anwendungssicht, Minimierung von: Durchlaufzeit** *(Zeit Starten Threads bis Ende)*, **Antwortzeit** *(Zeit Empfang Requests bis Antwort zur Verfügung steht)*, **Wartezeit** *(Zeit, die ein Thread in der Ready-Queue verbringt)*.
Aus Systemsicht, Maximierung von: Durchsatz *(Anzahl Threads, die pro Intervall bearbeitet werden)*, **Prozessor-Verwendung** *(Prozentsatz der Verwendung des Prozessors gegenüber der Nichtverwendung)*
Latenz durchschnittliche Zeit zwischen Auftreten und Verarbeiten eines Ereignisses. Im schlimmsten Fall tritt Ereignis dann auf, wenn der Thread gerade vom Prozessor entfernt wurde. Um Antwortzeit verringern, muss jeder Thread öfters ausgeführt werden, was jedoch zu mehr Thread-Wechsel und somit zu mehr Overhead führt. **Die Utilization nimmt also ab, wenn die Antwortzeit verringert wird.**

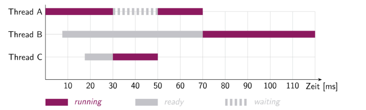
Idealfall: Parallele Ausführung *(Dient als **idealisierte Schranke**, besser geht nicht)*



FCFS-Strategie *(First Come, First Served)*
Nicht präemptiv: Threads geben den Prozessor nur ab, wenn sie auf waiting wechseln oder sich beenden.

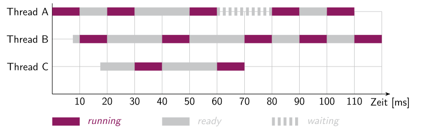


SJF-Strategie *(Shortest Job First)*
Scheduler wählt den Thread aus, der den **kürzesten** Prozessor-Burst hat. Bei gleicher Länge wird nach FCFS ausgewählt. Kann **kooperativ** oder **präemptiv** sein. Ergibt **optimale Wartezeit**, kann jedoch nur **korrekt implementiert** werden, wenn die Länge der Bursts **bekannt** sind.



Round-Robin: Zeitscheibe von etwa 10 bis 100ms. **FCFS**, aber ein Thread kann nur solange laufen, bis seine **Zeitscheibe erschöpft** ist, dann wird der in der **Queue hinten angehängt**.

Notiere: die Ready Queue pro Zeitscheibe, laufender Thread landet (wenn direkt Ready) ganz hinten in nächster Ready-Queue. Am besten pro Zeit die R.Q. notieren. Bei Wechsel zu `waiting` beginnt nächster Prozess früher, aber erhält keine zusätzliche Zeit.



Multi-Level Scheduling mit Feedback: Erschöpft ein Thread seine Zeitscheibe, wird seine Priorität um 1 verringert. Typischerweise werden die Zeitscheiben mit **niedrigerer Priorität grösser** und Threads mit **kurzen Prozessor-Bursts bevorzugt**. Threads in tiefen Queues dürfen zum Ausgleich länger am Stück laufen.

Prioritäten-basiert: Jeder Thread erhält **eine Nummer**, seine **Priorität**. Threads mit höherer Priorität werden vor Threads mit niedriger Priorität ausgewählt. Threads mit gleicher Priorität werden nach FCFS ausgewählt. Prioritäten je nach OS unterschiedlich.
Starvation: Thread mit niedriger Priorität wird immer **übergangen** und kann nie laufen. Abhilfe z.B. mit **Aging:** in bestimmten Abständen wird die Priorität um 1 erhöht.
Multi-Level Scheduling: Threads werden in **Level** aufgeteilt *(Priorität, Prozesstyp, Hinter-/Vordergrund)*, jedes Level hat eigene Ready-Queue und kann individuell gescheduled werden. *(zB. Timeslice/Queue)*

PRIORITÄTEN IN POSIX

Nice-Wert: Jeder Prozess hat einen Nice-Wert von -20 *(soll bevorzugt werden)* bis $+19$ *(nicht bevorzugt)*
`nice [-n increment] utility [argument...]`: Nice-Wert beim Start erhöhen oder verringern
`int nice (int i)`: Nice-Wert im Prozess erhöhen oder verringern. *(Addiert i zum Wert dazu.)*
`int getpriority (int which, id_t who)`: gibt den Nice-Wert von p zurück
`int setpriority (int which, id_t who, int prio)`: setzt den Nice-Wert.
- `which`: `PRIO_PROCESS`, `PRIO_PGRP` oder `PRIO_USER`
- `who`: ID des Prozesses, der Gruppe oder des Users)

Priorität bei Thread-Erzeugung setzen
Funktionen ohne `attr` bevor Thread gestartet wird *Funktionen geben POSIX int zurück*:
`pthread_getschedparam(pthread_t thread, int* policy, struct sched_param* param)`
`pthread_setschedparam(pthread_t thread, int policy, const struct sched_param* param)`

```c
pthread_attr_t a; pthread_attr_init (&a); // initialize attributes
struct sched_param p;
pthread_attr_getschedparam ( &a, &p ); // read parameter
// set p.sched_priority here... (code cut for brevity)
pthread_attr_setschedparam ( &a, &p ); // write modified parameters
pthread_create ( &id, &a, thread_function, argument );
pthread_attr_destroy ( &a ); // destroy attributes
```

MUTEXE UND SEMAPHORE

Jeder Thread hat seinen **eigenen** Instruction-Pointer und Stack-Pointer. Wenn Ergebnisse von der **Ausführungsreihenfolge** einzelner Instruktionen abhängen, spricht man von einer **Race Condition**. Threads müssen **synchronisiert** werden, damit keine **Race Condition** entsteht.

Critical Section: Code-Bereich, in dem Daten mit anderen Threads **geteilt** werden. Muss unbedingt synchronisiert werden.
Atomare Instruktionen: Eine atomare Instruktion kann vom Prozessor **unterbrechungsfrei** ausgeführt werden. ***Achtung:*** *Selbst einzelne Assembly-Instruktionen nicht immer atomar!*

Anforderungen an Synchronisations-Mechanismen:

Gegenseitiger Ausschluss *(Nur ein Thread darf in Critical Section sein)*, **Fortschritt** *(Entscheidung, wer in die Critical Section darf, muss in endlicher Zeit getroffen werden)*, **Begrenztes Warten** *(Thread wird nur n mal übergangen, bevor er in die Critical Section darf)*.

Implementierung: Nur **mit HW-Unterstützung** möglich. Es gibt zwei atomare Instruktionen:
Test-And-Set *(Setzt einen `int` auf 1 und returnt den vorherigen Wert: `test_and_set(int * target) {int value = *target; *target = 1; return value;}`)* und **Compare-and-Swap** *(Überschreibt einen `int` mit einem spezifizierten Wert, wenn dieser dem erwarteten Wert entspricht: `compare_and_swap (int *a, int expected, int new_a) {int value = *a; if (value == expected) { *a = new_a; } return value;}`)*.

SEMAPHORE

Enthält Zähler $z \geq 0$. Wird nur über `Post(v)` *(Erhöht z um 1)* und `Wait(v)` zugegriffen *(Wait: Wenn $z > 0$, verringert z um 1 und fährt fort. wenn $z = 0$, setzt den Thread in waiting, bis anderer Thread z erhöht)*.

*Alle diese Funktionen geben, gemäss **POSIX API** (thread library), den Status «int» zurück, weggekürzt.*

`sem_init (sem_t *sem, int pshared, unsigned int value):`
Initialisiert Semaphor vom Typ sem_t *sem_t möglich als globale Variable oder member von struct*, pshared=0 nur auf einem Prozessor. value=Startwert von Semaphor.
Verwendung globaler Semaphor über mehrere Prozesse:

```
sem_t sem; /*global*/
int main (..) { sem_init (&sem, 0, 4); /* pshared=0, Semaphor Wert =4 */ }
```

`sem_wait (sem_t *sem);` **`sem_post (sem_t *sem)`**: implementieren **Post +** und **Wait -**.
`sem_trywait (sem_t *sem);` **`sem_timedwait (sem_t *sem, const struct timespec *abs_timeout)`**: Sind wie `sem_wait`, aber **brechen ab**, falls Dekrement **nicht** durchgeführt werden kann. `sem_trywait` bricht sofort ab, `sem_timedwait` nach der angegebenen Zeitdauer.
`sem_destroy (sem_t *sem)`: **Entfernt Speicher**, den das OS mit `sem` **assoziiert** hat.

```
semaphore free = n; semaphore used = 0; // Beispiel mit 1 Producer und 1 Consumer

// Producer                                   // Consumer
while (1) {                                   while (1) {
  WAIT (free); //warte auf Platz in Queue       // Warte, falls Producer zu langsam
  produce_item (&buffer[w], ...);               WAIT (used); // Hat es Elem. in Queue?
  POST (used); // 1 Element mehr in Queue       consume (&buffer[r]);
  w = (w+1) % BUFFER_SIZE;                      POST (free); // 1 Element weniger in Q
}                                               r = (r+1) % BUFFER_SIZE;
                                              }
```

MUTEXE

Ein Mutex hat einen **binären Zustand z**, der nur durch zwei Funktionen verändert werden kann: **Acquire** *(Wenn $z = 0$, setze z auf 1 und fahre fort. Wenn $z = 1$, blockiere den Thread, bis $z = 0$)*, **Release** *(Setzt $z = 0$)*. Auch als non-blocking-Funktion: `int pthread_mutex_trylock (pthread_mutex_t *mutex)`

```
// Beispiel Initialisierung                  // Beispiel Verwendung in Threads
pthread_mutex_t mutex; // global             void * thread_function (void * args) {
int main() {                                   while (running) { ...
pthread_mutex_init (&mutex, 0);                  // Enter critical section:
// run threads & wait for them to finish         pthread_mutex_lock (&mutex);
pthread_mutex_destroy (&mutex); }                // Leave critical section:
                                                 pthread_mutex_unlock (&mutex);...}}
```

Priority Inversion: Ein **hoch-priorisierter** Thread C wartet auf eine Ressource, die von einem **niedriger priorisierten** Thread A **gehalten** wird. Ein Thread mit Priorität zwischen diesen beiden Threads erhält den Prozessor. Kann mit **Priority Inheritance** gelöst werden: Die Priorität von A wird temporär auf die Priorität von C gesetzt, damit der Mutex schnell wieder freigegeben wird.

SIGNALE, PIPES UND SOCKETS

SIGNALE

Signale ermöglichen es, einen Prozess **von aussen** zu unterbrechen, wie ein **Interrupt**. **Unterbrechen** des gerade laufenden Prozesses/Threads, Auswahl und Ausführen der **Signal-Handler-Funktionen**, **Fortsetzen** des Prozesses. Werden über ungültige Instruktionen oder Abbruch auf Seitens Benutzer ausgelöst. Jeder Prozess hat pro Signal einen Handler.

Mit `kill` kann ein Signal, mit einem dieser Default-Handler, gesendet werden:
Abnormal-Terminate-Handler | Beendet Programm + Core Dump (für Programmfehler)
`SIGFPE` *(Fehler in arithmetischen Operation)*, `SIGILL` *(Ungültige Instruktion)*,
`SIGSEGV` *(Ungültiger Speicherzugriff)*, `SIGSYS` *(Ungültiger Systemaufruf)*
Terminate-Handler | Beendet das Programm (Prozess abbrechen)
`SIGKILL` *(**nicht überschreibbar**)* `SIGTERM` *(Normale höfliche Anfrage an den Prozess, sich zu beenden)*,
`SIGINT` *(Nachdrücklichere Aufforderung an Prozess, sich zu beenden)*, `SIGQUIT` *(Wie `SIGINT`, aber anormale Terminierung `Ctrl-\`)*, `SIGABRT` *(Wie `SIGQUIT`, aber vom Prozess an sich selber (Programmierfehler bemerkt))*,
Ignore-Handler | ignoriert das Signal
`SIGTSTP` *(Versetzt den Prozess in den Zustand **stopped**, ähnlich wie **waiting** `Ctrl-Z`)*,
`SIGSTOP` *(Wie `SIGTSTP`, aber **nicht überschreibbar**)*, `SIGCONT` *(Setzt den Prozess fort (`fg`))*

`int sigaction(int signal, struct sigaction *new, struct sigaction *old):`
Definiert Signal-Handler für `signal`, wenn `new` $\neq 0$. *(Eigene Signal-Handler definiert via `sigaction` struct: `sa_handler`: Zu callende Funktion, `sa_mask`: Blockierte Signale während Ausführung, bearbeitet nur durch `sig*set()`-Funktionen: `sigemptyset, sigfillset, sigaddset, sigdelset, sigismember`)*
sigaction = transp. Datenstruktur, sigset_t = opake Datenstr. + implementationsabhängig

PIPES

geöffnete Datei: entspricht einem **Eintrag in der File-Descriptor-Tabelle (FDT)** im Prozess. Zugriff über **File-API** *(`open, close, read, write, …`)*. Das OS speichert **je Eintrag der Prozess-FDT** einen **Verweis auf die globale FDT**.
`fork()`: die FDT wird auch kopiert.
`int dup (int source_fd); int dup2 (int source_fd, int destination_fd)`: Duplizieren und returnen den FD `source_fd`. `dup` alloziert neuen FD, `dup2` überschreibt `destination_fd`.

Umleiten des Ausgabestreams

```
int fd = open("log.txt", ...);   int pid = fork();
if (pid == 0) { // child
  dup2(fd, 1); // duplicate fd for log.txt as standard output
  // e.g. load new image with exec*, fd's remain
} else { /* parent */ close (fd); }
```

(Eine Datei muss nur `open` & `close` & `read`/`write` unterstützen)
Pipe: «Datei» im Hauptspeicher, die über zwei FD verwendet wird:
read end zum Lesen-, **write end** zum Schreiben in die Pipe *kein `lseek`*.
Daten, die in **write end** geschrieben werden, können aus **read end** genau **einmal** gemäss **FIFO** gelesen werden. Pipes erlauben Kommunikation über Prozess-Grenzen hinweg. *Ist unidirektional*
`int pipe (int fd [2]); // equivalent to int pipe (int *fd)`

```
int fd[2]; // 0 = read, 1 = write
pipe (fd);
int id = fork();

if (id == 0) { // Child thread
  close (fd [1]); // don't use write end
  char buffer [BSIZE];
  int n = read (fd[0], buffer, BSIZE);
} else { // Parent thread
  close (fd[0]); // don't use read end
  char * text = "I <3 segfaults";
  write (fd[1], text, strlen(text) + 1);
}
```

Pipe lebt nur so lange, wie mind. ein Ende geöffnet ist. Alle Read-Ends geschlossen → `SIGPIPE` an Write-End. Mehrere Writes können zusammengefasst werden.
Lesen mehrere Prozesse dieselbe Pipe, ist unklar, wer die Daten erhält.

- keine Daten in der Pipe → read blockiert, bis Daten hineingeschrieben werden
- keine Daten in der Pipe + kein geöffnetes Write-End → read gibt 0 zurück (EOF)
- Default Pipe Grösse = 16 Pages, meistens 64KB

`int mkfifo (const char *path, mode_t mode):` Erzeugt eine Pipe **mit Namen und Pfad** im Dateisystem. Hat via `mode` Permission Bits wie normale Datei. Lebt unabhängig vom erzeugenden Prozess, je nach System auch über Reboots hinweg. Muss explizit mit `unlink` gelöscht werden.

SOCKETS

Ein Socket **repräsentiert einen Endpunkt auf einer Maschine**. Kommunikation findet im Regelfall zwischen zwei Sockets statt *(UDP, TCP über IP sowie Unix-Domain-Sockets)*.

Sockets benötigen für Kommunikation einen Namen: *(IP: IP-Adresse & Portnummer)*
`int socket(int domain, int type, int protocol):` **Erzeugt** einen neuen Socket als «Datei». Socket sind nach Erzeugung zunächst **unbenannt**. Alle Operationen blockieren per default.
`Domain` *(`AF_UNIX, AF_INET`)*, `type` *(`SOCK_DGRAM, SOCK_STREAM`)*, `protocol` *(System-spezifisch, 0 = Default-Protocol)*
Client: `connect` *(Verbindung unter Angabe einer Adresse aufbauen)*, `send / write` *(Senden von Daten, $0 - \infty$ mal)*, `recv / read` *(Empfangen von Daten, $0 - \infty$ mal)*, `close` *(Schliessen der Verbindung)*
Server: `bind` *(Festlegen einer nach aussen sichtbaren Adresse)*, `listen` *(Bereitstellen einer Queue zum Sammeln von Verbindungsanfragen von Clients)*, `accept` *(Erzeugen einer Verbindung auf Anfrage von Client)*, `recv / read` *(Empfangen von Daten, $0 - \infty$ mal)*, `send / write` *(Senden von Daten, $0 - \infty$ mal)*, `close` *(Schliessen der Verbindung)*

```
struct sockaddr_in ip_addr;
ip_addr.sin_port = htons (443); // default HTTPS port
inet_pton (AF_INET, "192.168.0.1", &ip_addr.sin_addr.s_addr);
// IP address in memory: 0xC0 0xA8 0x00 0x01, Port in memory: 0x01 0xBB
```

`htons()` konvertiert 16 Bit von Host-Byte-order *(LE)* zu Network-Byte-Order *(BE)*, `htonl()` 32 Bit. `ntohs()` und `ntohl()` sind Gegenstücke. `inet_pton()` konvertiert protokoll-spezifische Adresse von String zu Network-BO. `inet_ntop()` ist das Gegenstück *(network-to-presentation)*.
`int bind (int socket, const struct sockaddr *local_address, socklen_t addr_len):` **Bindet** den Socket an die angegebene, unbenutzte lokale Adresse, wenn noch nicht gebunden. Blockiert, bis der Vorgang abgeschlossen ist.
`int connect (int socket, const struct sockaddr *remote_addr, socklen_t addr_len):` **Aufbau** einer Verbindung. Bindet den Socket an eine neue, unbenutzte lokale Adresse, wenn noch nicht gebunden. Blockiert, bis Verbindung steht oder ein Timeout eintritt.
`int listen (int socket, int backlog):` **Markiert** den Socket als «bereit zum Empfang von Verbindungen». Erzeugt eine Warteschlange, die so viele Verbindungsanfragen aufnehmen kann, wie `backlog` angibt.
`int accept (int socket, struct sockaddr *remote_address, socklen_t address_len):` **Wartet**, bis Verbindungsanfrage in der Warteschlange eintrifft. Erzeugt einen neuen Socket und bindet ihn an eine neue lokale Adresse. Die Adresse des Clients wird in `remote_address` geschrieben. Der neue Socket kann keine weiteren Verbindungen annehmen, der bestehende schon.

Typisches Muster für Server

```
int server_fd = socket ( ... ); bind (server_fd, ...); listen (server_fd, ...);
while (running) {
  int client_fd = accept (server_fd, 0, 0);
  delegate_to_worker_thread (client_fd); // will call close(client_fd)
}
```

```
ssize_t send (int socket, const void *buffer, size_t length, int flags);
/* wie write: */ ssize_t write (int fd, void *buffer, size_t n)
ssize_t recv (int socket, void *buffer, size_t length, int flags);
/* wie read: */ ssize_t read (int fd, void *buffer, size_t n)
send (fd, buf, len, 0); // Aufruf Beispiel
```

Senden und Empfangen von Daten

Erweitern `read()` bzw. `write()` um Socket-Funktionalitäten durch den `flags`-Parameter. Puffern der Daten ist Aufgabe des Netzwerkstacks.

`int close (int socket)`: Schliesst Socket für den aufrufenden Prozess. Hat anderer Prozess den Socket geöffnet, bleibt Verbindung bestehen. Gegenseite wird nicht benachrichtigt. Gegenseite erhält bei einem read kein EOF und verlässt die Lese-Schleife nicht.
`int shutdown (int socket, int mode)`: Schliesst Socket für alle Prozesse und baut die entsprechende Verbindung ab. `mode`: `SHUT_RD` *(Keine Lese-Zugriffe mehr)*, `SHUT_WR` *(Keine Schreib-Zugriffe mehr)*, `SHUT_RDWR` *(Keine Lese- oder Schreib-Zugriffe mehr)*

MESSAGE PASSING UND SHARED MEMORY UND PIPES

Prozesse sind voneinander isoliert, müssen jedoch trotzdem miteinander interagieren.

Shared Memory ist schneller zu realisieren, aber schwer wartbar. **Message-Passing** erfordert mehr Engineering-Aufwand, schlussendlich aber in Mehr-Prozessor-Systemen bald performanter.

MESSAGE-PASSING

ist ein Mechanismus mit zwei Operationen:
- **Send** *(Kopiert die Nachricht aus dem Prozess: `send (message)`)*,
- **Receive:** *(Kopiert die Nachricht in den Prozess: `receive (message)`)*.

Unterscheidungskritierien

Dabei können Implementierungen nach verschiedenen Kriterien unterschieden werden

Feste oder variable Nachrichtengrösse: feste Nachrichtengrösse ist einfacher zu implementieren, aber umständlicher zu verwenden als variable Nachrichtengrösse.
Direkte Kommunikation: Kommunikation nur zwischen genau zwei Prozessen, Sender muss Empfänger kennen. Es gibt **symmetrische direkte Kommunikation** *(Empfänger muss Sender auch kennen (oder Alias))* und **asymmetrische direkte Kommunikation** *(Empfänger muss Sender nicht kennen, sondern erhält die ID in einem Out-Parameter id `receive (id, message)`)*.
Indirekte Kommunikation: Prozess sendet Nachricht an **Mailboxen, Ports oder Queues**. Empfänger empfängt aus diesem Objekt. Beide Teilnehmer müssen die gleiche(n) Mailbox(en) kennen.
Weitere: synchrone oder asynchrone Kommunikation, Pufferung, mit oder ohne Prioritäten für Nachrichten

Indirekte Kommunikation
Lebenszyklus Queue: Wenn diese Queue einem Prozess gehört, lebt sie solange wie der Prozess. Wenn sie dem OS gehört, muss das OS das Löschen übernehmen.
Synchronisation: Blockierendes Senden *(Sender wird solange blockiert, bis die Nachricht vom Empfänger empfangen wurde)*, Nicht-blockierendes Senden *(Sender sendet Nachricht und fährt sofort weiter)*, Blockierendes Empfangen *(Empfänger wird blockiert, bis Nachricht verfügbar)*, Nicht-blockierendes Empfangen *(Empfänger erhält Nachricht, wenn verfügbar, oder 0)*
Rendezvous: Sind Empfang und Versand **beide blockierend**, kommt es zum Rendezvous, sobald beide Seiten ihren Aufruf getätigt haben. **Impliziter Synchronisationsmechanismus**.

```
// Producer                               // Consumer
message msg;                              message msg;
open(Q);                                  open(Q);
while(1) {                                while(1) {
  produce_next(&msg);                       receive(Q, &msg); // blocked until rec.
  send(Q, &msg); // blocked until sent      consume_next(&msg);
}                                         }
```

Pufferung: Keine *(Queue-Länge ist 0, Sender muss blockieren)*, **Beschränkte** *(Maximal n Nachrichten, Sender blockiert, wenn Queue voll ist.)*, **Unbeschränkte** *(Beliebig viele Nachrichten, Sender blockiert nie)*.
Prioritäten: In manchen Systemen können Nachrichten mit **Prioritäten** versehen werden. Der Empfänger holt die Nachricht mit der **höchsten Priorität zuerst** aus der Queue.

POSIX Message-Passing = OS Message Queue API

Nutzt OS-Message-Queues mit **variabler Länge**, haben mind. 32 Prioritäten und können **synchron und asynchron** verwendet werden, indirekte Kommunikation.
`mqd_t mq_open (const char *name, int flags, mode_t mode, struct mq_attr *attr)`: **Öffnet** eine Message-Queue mit systemweitem `name`, returnt Message-Queue-Descriptor. *(`name` mit «/» beginnen, `flags` & `mode` wie bei Dateien, `mq_attr`: Div. Konfigs & Queue-Status, R/W mit `mp_getattr/mq_setattr`)*
`int mq_close (mqd_t queue)`: **Schliesst** die Queue mit dem Descriptor `queue` für diesen Prozess.
`int mq_unlink (const char *name)`: **Entfernt** die Queue mit dem Namen `name` aus dem System. **Name** wird **sofort entfernt** und Queue kann anschliessend **nicht mehr geöffnet** werden.
`int mq_send (mqd_t queue, const char *msg, size_t length, unsigned int priority)`: **Sendet** die Nachricht, die an Adresse `msg` beginnt und `length` Bytes lang ist, in die `queue`.
`int mq_receive (mqd_t queue, const char *msg, size_t length, unsigned int *priority)`: **Kopiert** die nächste Nachricht aus der Queue in den Puffer, der an Adresse `msg` beginnt und `length` Bytes lang ist. **Blockiert**, wenn die Queue **leer** ist.

Message-Queues	Pipes
– bidirektional – Daten sind in einzelnen Messages organisiert – beliebiger Zugriff – Haben immer einen Namen	– unidirektional – übermittelt Bytestrom an Daten – FIFO-Zugriff – Müssen keinen Namen haben

SHARED MEMORY

Frames des Hauptspeichers werden **zwei (oder mehr) Prozessen** P_1 und P_2 **zugänglich** gemacht. In P_1 wird Page V_1 auf einen Frame F abgebildet. In P_2 wird Page V_2 auf **denselben** Frame F abgebildet. Beide Prozesse können **beliebig** auf dieselben Daten zugreifen. Im Shared Memory müssen **relative Adressen** verwendet werden.

POSIX API
Das OS benötigt eine «Datei» S, das Informationen über den gemeinsamen Speicher verwaltet und eine **Mapping Table** je Prozess.

POSIX (mode für shm_open, mq_open): `S_IRWXU = 0700`, `S_IWUSR = 0200`, `S_IRGRP = 0040`, `S_IXOTH = 0001`. Werden mit | verknüpft.
`int fd = shm_open ("/mysharedmemory", O_RDWR | O_CREAT, S_IRUSR | S_IWUSR)`: Erzeugt (falls nötig) und öffnet Shared Memory `/mysharedmemory` zum Lesen und Schreiben.
`int ftruncate (int fd, offset_t length)`: **Setzt** Grösse der «Datei». Muss **zwingend** nach SM-Erstellung gesetzt werden, um entsprechend viele Frames zu allozieren.
Wird für Shared Memory mit ganzzahligen Vielfachen der Page-/Framegrösse verwendet.
`int close (int fd)`: **Schliesst** «Datei». Shared Memory **bleibt aber im System**.
`int shm_unlink (const char * name)`: **Löscht** das Shared Memory mit dem `name`. *(bleibt vorhanden, solange noch von Prozess geöffnet)*
`int munmap (void *address, size_t length)`: **Entfernt** das Mapping.

```
void * address = mmap( // maps shared memory into virt. address space of process
  0,      // void *hint_address (0 because nobody cares)
  size_of_shared_memory,  // size_t length (same as used in ftruncate)
  PROT_READ | PROT_WRITE, // int protection (never use execute)
  MAP_SHARED,    // int flags
  fd,     // int file_descriptor
  0       // off_t offset (start map from first byte)
);
```

UNICODE

ASCII - AMERICAN STANDARD CODE FOR INFORMATION INTERCHANGE

Hex: `1010=A, 1011=B, 1100=C, 1101=D, 1110=E, 1111=F`
Hat **128 definierte Zeichen** *(erste Hexzahl = Zeile ↓, zweite Hexzahl = Spalte →, d.h. 41_h = A)*.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH 1	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SB	ESC	FS	GS	RS	US
2	␣	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Codepages: unabhängige Erweiterungen auf 8 Bit. Jede ist unterschiedlich und nicht erkennbar.
Unicode: Hat zum Ziel, einen eindeutigen Code für **jedes vorhandene Zeichen** zu definieren. $D8\,00_h$ bis $DF\,FF_h$ sind wegen UTF-16 keine gültigen Code-Points.
Code-Points (CP oder P): Nummer eines Zeichen - «welches Zeichen?» in der Unicode Tabelle
Code-Unit (CU): Einheit, um Zeichen in einem Encoding darzustellen *(bietet den Speicherplatz)*
P_i = *i-tes Bit des unkodierten CPs*, U_i = *i-tes Code-Unit des kodierten CPs*, B_i = *i-tes Byte des kodierten CPs*

UTF-32

Jede CU ist die ganze **32 Bit** gross, jeder CP ist eine CU. Wird häufig intern in Programmen verwendet. Obere 11 Bits oft «zweckentfremdet». Endianness gibt an, mit welchem Ende die Folge anfängt:
Big-Endian: B3 B2 B1 B0 *CP aus Unicode-Tabelle*,
Little-Endian: B0 B1 B2 B3

UTF-8

Jede CU umfasst 8 Bit, ein CP benötigt 1 bis 4 CUs. Standard für Webpages. Echte Erweiterung von ASCII. Jedes B (Byte) = U (Code-Unit). Maximal 4 Bytes.

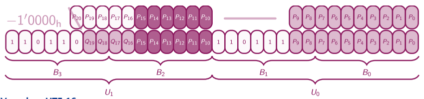
Code-Point in	U_3	U_2	U_1	U_0	signifikant
0_h - $7F_h$				$0xxx\ xxxx_b$	7 bits
80_h - $7\,FF_h$			$110x\ xxxx_b$	$10xx\ xxxx_b$	11 bits
$8\,00_h$ - $FF\,FF_h$		$1110\ xxxx_b$	$10xx\ xxxx_b$	$10xx\ xxxx_b$	16 bits
$1\,00\,00_h$ - $10\,FF\,FF_h$	$1111\ 0xxx_b$	$10xx\ xxxx_b$	$10xx\ xxxx_b$	$10xx\ xxxx_b$	21 bits

Vorgehen UTF-8
ä: $P = E4_h =$
1. in Binär umwandeln 1110 0100
2. von rechter Seite her die Bits aufteilen und Separator-Bits korrekt einfügen:
   1 1|10 0100 → 110 0 0011 10 10 0100
3. am Schluss zurück in Hex umwandeln: C3 A4

UTF-16

Jede CU umfasst 16 Bit, ein CP benötigt 1 oder 2 CUs. Die 2 CUs werden **Surrogate Pair** genannt, U_0: high surrogate, U_1: low surrogate. Bei **2 Bytes** *(1 CU)* wird direkt gemappt und vorne mit Nullen aufgefüllt, *CP in [0 - FFFF] ohne [D800 - DFFF]*. Bei **4 Bytes** sind $D8\,00_h$ bis $DF\,FF_h$ *(Bits 17-21)* wegen dem Separator **ungültig** und müssen «umgerechnet» werden.

BE: U0 = B1 B0, **BE**: U1 U0 = B3 B2 B1 B0,
LE: U0 = B0 B1 **LE**: U1 U0 = B2 B3 B0 B1



Vorgehen UTF-16
kleiner als FFFF: ist BE, Bytes anschreiben (1 Byte = 2 Hex. Zeichen)
grösser als FFFF: Beispiel: Encoding von U+10'437 = 10 '4 37$_h$ = (𐐷)
1. minus 1 00 00$_h$ rechnen (die 0 behalten!) 00 '4 37$_h$
2. in Binär umwandeln 0000 0000 0100 0011 0111$_b$
3. untere (rechts) 10 Bits nehmen und rechts schreiben → 00 0011 0111
4. Surrogat 110111 links von den 10 Bits → 11 0111 00 0011 0111
5. restliche (linker Teil) Bits links vom Surrogat schreiben (diese: 00 0000 0001)
   → 00 0000 0001 1101 1100 0011 0111
6. ganz links die 110110 schreiben → 11 0110 00 0000 0001 1101 1100 0011 0111
7. zurück zu Hex umwandeln
   110 1-10 0 0-00 00-0 001- 110 1-11 00-0 011- 0111 → D8 01 DC 37$_h$
8. Endianness BE = D8 01 DC 37$_h$, LE = 01 D8 37 DC$_h$

ENCODING-BEISPIELE

Zeichen	Code-Point	UTF-32BE	UTF-32LE	UTF-8	UTF-16BE	UTF-16LE
A	41_h	$00\,00\,00\,41_h$	$41\,00\,00\,00_h$	41_h	$00\,41_h$	$41\,00_h$
ä	$E4_h$	$00\,00\,00\,E4_h$	$E4\,00\,00\,00_h$	$C3\,A4_h$	$00\,E4_h$	$E4\,00_h$
α	$3\,B1_h$	$00\,00\,03\,B1_h$	$B1\,03\,00\,00_h$	$CE\,B1_h$	$03\,B1_h$	$B1\,03_h$
ặ	$1E\,B7_h$	$00\,00\,1E\,B7_h$	$B7\,1E\,00\,00_h$	$E1\,BA\,B7_h$	$1E\,B7_h$	$B7\,1E_h$
𐌰	$1\,03\,30_h$	$00\,01\,03\,30_h$	$30\,03\,01\,00_h$	$F0\,90\,8C\,B0_h$	$D8\,00\,DF\,30_h$	$00\,D8\,30\,DF_h$

*Bei LE / BE werden nur die Zeichen **innerhalb** eines Code-Units vertauscht, nicht die Bytes an sich.*

DATEISYSTEME

VERGLEICH FAT, NTFS, EXT2

FAT	Ext2	NTFS
– Verzeichnis enthält alle Daten über Datei – Datei in 1 einzigen Verzeichnis – Keine Hard-Links möglich	– Dateien durch Inodes beschrieben – Kein Link von Datei zurück zum Verzeichnis	– Dateien durch File-Records beschrieben – Verzeichnis = Namen und Link auf Datei – Link zum Verzeichnis und Name sind in Attribut

BLOCKGRUPPE

Eine Blockgruppe besteht aus **mehreren aufeinanderfolgenden Blöcken** bis zu 8 mal der Anzahl Bytes in einem Block.
Layout: Block 0 *(Kopie des Superblocks)*, **Block 1 bis n** *(Kopie der Gruppendeskriptorentabelle)*, **Block $n+1$** *(Block-Usage-Bitmap mit einem Bit je Block der Gruppe)*, **Block $n+2$** *(Inode-Usage-Bitmap mit einem Bit je Inode der Gruppe)*, **Block $n+3$ bis $n+m+2$** *(Tabelle aller Inodes in dieser Gruppe)*, **Block $n+m+3$ bis Ende der Gruppe** *(Blöcke der eigentlichen Daten)*
Superblock: Enthält **alle Metadaten** über das Volume *(Anzahlen, Zeitpunkte, Statusbits, Erster Inode, …)* immer an Byte 1024, wegen möglicher Bootdaten vorher.
Sparse Superblock: Kopien des Superblocks werden nur in Blockgruppe 0, 1 und allen reinen Potenzen von 3, 5 oder 7 gehalten *(Sehr hoher Wiederherstellungsgrad, aber deutlich weniger Platzverbrauch)*.
Gruppendeskriptor: Beschreibung einer Blockgruppe. *(Blocknummer des Block-Usage-Bitmaps, Blocknummer des Inode-Usage-Bitmaps, Nummer des ersten Blocks der Inode-Tabelle, Anzahl freier Blöcke und Inodes in der Gruppe, Anzahl der Verzeichnisse in der Gruppe)*
Gruppendeskriptortabelle: Tabelle mit Gruppendeskriptor pro Blockgruppe im Volume. Folgt direkt auf Superblock(-kopie). $32 \cdot n$ Bytes gross. Anzahl Sektoren $= (32 \cdot n)/\text{Sektorgrösse}$
Verzeichnisse: Enthält **Dateieinträge** mit variabler Länge von 8 - 263 Byte *(4B Inode, 2B Eintraglänge, 1B Dateinamenlänge, 1B Dateityp, 0 - 255B Dateiname aligned auf 4B)*. Defaulteinträge: «.» und «..»
Links: Es gibt **Hard-Links** *(gleicher Inode, verschiedene Pfade: Wird von verschiedenen Dateieinträgen referenziert)* und **Symbolische Links** *(Wie eine Datei, Datei enthält Pfad anderer Datei)*.

BLOCK

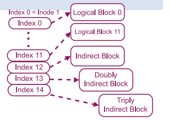
Ein Block besteht aus **mehreren aufeinanderfolgenden Sektoren**. Das gesamte Volume ist in **Blöcke aufgeteilt** und Speicher wird **nur in Form von Blöcken** alloziert. Ein Block enthält nur Daten einer **einzigen Datei**. Es gibt **Logische Blocknummern** (Blocknummer vom Anfang der Datei aus gesehen, wenn Datei eine ununterbrochene Abfolge von Blöcken wäre) und **Physische Blocknummern** (Tatsächliche Blocknummer auf dem Volume).
File-Holes: Bereiche in der Datei, in der **nur Nullen** stehen. Ein solcher Block wird **nicht alloziert**.

EXT2-DATEISYSTEM

Partition *(Ein Teil eines Datenträgers, wird selbst wie ein Datenträger behandelt.)*, **Volume** *(Ein Datenträger oder eine Partition davon.)*, **Sektor** *(Kleinste logische Untereinheit eines Volumes. Daten werden als Sektoren transferiert. Grösse ist von HW definiert. Enthält Header, Daten und Error-Correction-Codes.)*,
Format *(Layout der logischen Strukturen auf dem Datenträger, wird vom Dateisystem definiert.)*
Inode Aufteilung ext2: $15\ \text{Blocknummern} \cdot \text{Inodenummer-Grösse} = 15 \cdot 4$ Byte = 60 Byte
Referenzen-Block: Synonym für indirekter Block!
Anzahl direkte Blöcke: 12 (bis und mit Block B_h), 1024 indirekt Blöcke möglich

INODES

Enthält **alle Metadaten** über die Datei, **ausser Namen oder Pfad** (Grösse, Anzahl der verwendeten Blöcke, Erzeugungs-, Zugriffs-, Modifikations- und Löschzeit, Owner-ID, Group-ID, Flags, Permission Bits).
Inode **verweist auf die Blöcke**, die **Daten für eine Datei** enthalten. Enthält ein Array `i_block` (Blockliste) mit 15 Einträgen zu je 32 Bit.



Inodenummer-Grösse (im Verzeichnis): 4 Byte *immer*
Lokalisierung: Alle Inodes aller Blockgruppen gelten als **eine grosse Tabelle**. Startet mit 1.
Erzeugung: Neue Verzeichnisse werden in der Blockgruppe angelegt, die von allen Blockgruppen mit **überdurchschnittlich vielen freien Inodes** die **meisten Blöcke frei** hat, Dateien in der Blockgruppe des Verzeichnis oder nahen Gruppen. Bestimmung anhand **Inode-Usage-Bitmaps**.

EXT4

Extent Tree Header Aufbau (12B)
2 Byte Magic Number $F30A_h$, **2B** Anz. Einträge, die direkt auf den Header folgen, **2B** Anz. Einträge, die maximal auf den Header folgen können, **2B** Tiefe, **4B** reserviert. **Tiefe**: **0**: Einträge sind Extent-Blöcke, **>=1**: Einträge sind Index Nodes.

INDEX-BLÖCKE (EXT4) INDEX-NODE / EXTENT

- Jeder Index-Block ist Teil vom Baum, haben Grösse von einem Blöck *ausser Inode*
- Enthält Referenzen auf Index Nodes *Block mit Index-Nodes* oder Extents *Index-Node*

Inode = Wurzel des Extent Baums
- Tiefe im Extent Tree Header bis 5 möglich *deshalb Maximal 2^{32} = 4G Blöcke pro Datei*.
- 256 Bytes mit Metadaten und `i_block`, der `i_block` ist 60 Byte gross
- `i_block` ist **Index-Node** oder **Extent**.
  - Bei Extent: $(1\ \text{Header} + 4\ \text{Extents}) \cdot \text{Extent-Grösse} = 5 \cdot 12$ Byte = 60 Byte

Index-Node Tiefe > 0 innerer Knoten
- Nötig ab mehr als $4 \cdot 340 = 1'360$ Extents bei 4KB Blockgrösse.
- Header mit Tiefe grösser als 0 + Index-Einträge
- Vereinfacht die Suche nach logischen Blocknummern im Baum
- **Index-Einträge** *verweist auf Kindknoten*: kleinste logische Blocknummer der Kindknoten, Nummer des physischen Index-Blocks

Extent Tiefe == 0 Blattknoten
- Der Block enthält am Anfang einen Header (wie im Inode, aber **Tiefe = 0**) danach die Extent-Einträge (max. 340 bei 4 KB Blockgrösse)
- **Extent-Einträge** beschreiben **Intervall physisch konsekutiver Blöcke**, **12 Byte**: **4B** erste logische Blocknummer, **6B** erste physische Blocknummer, **2B signed** Anzahl Blöcke

EXT2 + EXT4 FORMELN UND ZAHLEN

4096	2048	1024	512	256	128	64	32	16	8	4	2	1
2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
$10\,00_h$	$8\,00_h$	$4\,00_h$	$2\,00_h$	$1\,00_h$	80_h	40_h	20_h	10_h	8_h	4_h	2_h	1_h

Dezimal zu Hex: **1K** $= 2^{10} = 100\,0000\,0000_b = 400_h$, **4K** $= 2^{12} = 1\,0000\,0000\,0000_b = 1000_h$, **1G** = 2^{30} = $4000\,0000_h$, **12 Byte** = 100 Bit, 24 Bit=3 Byte, 32 Bit=4 Byte, 40 Bit=5 Byte

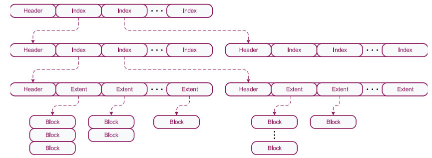
Datenstrukturen Grössen ext2 und ext4
Inodes *fixe Grösse je Volume*: **ext2**: Zweiterpotenz mind. 128 B, **ext4**: mind. 256 B und max. 1 Block
Gruppendeskriptoren: **ext2**: 32 B, **ext4**: 64 B
Blockgrösse: **ext2**: 1KB, 2KB oder 4KB (normal), **ext4**: bis 64 KB

ext2 Formeln
Anz. Referenzen pro Referenzen-Block R: Blockgrösse / 4B *Inodenummer-Grösse* = 4KB / 4B = 1K = 400_h
Anzahl indirekte Blöcke: R (siehe Referenzen pro Ref.Block) = 1K = 400_h
Anzahl doubly indirekte Blöcke: $R^2 = 1K^2 = 2^{10*2} = 2^{20} = 1M$ = $10\,0000_h$
Anzahl triply indirekte Blöcke: $R^3 = 2^{30} = 1G$ = $4000\,0000_h$
letzte Blocknummer (physisch): (Startindex + Anzahl Blöcke) - 1 = $A000_h$ - 1 = $9FFF_h$

Inode ist bekannt Formeln
Inode zu Index der Blockgruppe: (Inode-1) / Anz. Inodes pro Gruppe
Inode zu Index des Inodes innerhalb Blockgruppe: (Inode-1) % Anz. Inodes pro Gruppe

BEISPIEL

3MB grosse, konsekutiv gespeicherte Datei, 4KB Blöcke ab Block 2000_h
(In-)direkte Block-Adressierung (ext2)
Erster Referenzen-Block (Metadaten) 800_h, danach fortlaufend. Sonst nach letzter Adresse bei: 2300_h
3 MB = $3 \cdot 2^{20}$B, 4 KB = 2^{12}B, $3 \cdot 2^{20-12} = 3 \cdot 2^8 = 300_h$ Blöcke, von 2000_h bis $22FF_h$
$0 \mapsto 2000_h$, $1 \mapsto 2001_h$, …, $B_h \mapsto 200B_h$, $C_h \mapsto 800_h$ *(indirekter Block)*
$800.0_h \mapsto 200C_h$, $800.1_h \mapsto 200D_h$, …, $800.22F3_h \mapsto 22FF_h$
Extent Trees
Header: $0 \mapsto (1\ \text{Anz. Einträge}, 0\ \text{Tiefe})$
Extent: $1 \mapsto (0\ \text{1. logischer Block}, 20\,00_h\ \text{1. physischer Block}, 3\,00_h\ \text{Anz. Blöcke})$

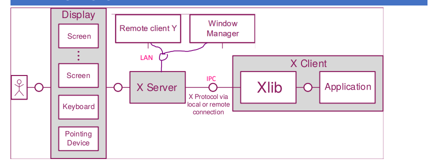


- Bei Berechnung von genutzten Metadaten, doppelt indirekte Blöcke aufzeichnen, damit ein «Wurzelblock» nicht vergessen geht.

JOURNALING (EXT3, EXT4)

Journaling verringert Zeit für Überprüfung von Inkonsistenzen *z.B. Stromausfall/Crash*.
Dateierweiterung Änderungen: neue Blöcke, Inode, Block-Usage-Bitmaps, Counter freier/benutzter Blöcke, Daten in Datei.
Journal: Reservierte Datei, Metadaten für Journaling. Typischerweise: Inode 8, 128 MB, bestenfalls 1 Extent. Schnelles schreiben: konsekutive Blöcke.
Transaktion: Folge von Einzelschritten, möglichst gesamtheitlich durchführen von Dateisystem.
Journaling: Transaktion ins Journal schreiben. *Daten nach Commit aus Journal entfernen*
Comitting: (Dateiinhalte +) Metadaten an korrekter Position schreiben
Journal Replay: Transaktionen im Journal nach Neustart noch einmal ausführen oder Fehler finden.
(Full) Journal: Metadaten und Datei-Inhalte zuerst ins Journal dann von dort in den Speicher, sehr sicher aber langsam, +Datensicherheit -Geschwindigkeitseinbussen
Ordered: 1. Transaktion (Metadaten) ins Journal, 2. Dateiinhalte direkt schreiben, 3. Commit ausführen (Metadaten), +Dateien enthalten richtigen Inhalt nach Commit - bisschen langsamer als Writeback -Blockaden bei vielen Transaktionen möglich
Writeback: Zuerst Metadaten ins Journal, beliebige Reihenfolge von Commit und Datei schreiben, +sehr schnell *keine Synchronisation Reihenfolge*, -Dateien können Datenmüll enthalten

X WINDOW SYSTEM



Setzt **Grundfunktionen der Fensterdarstellung**. Ist austauschbar, realisiert Netzwerktransparenz. Plattformunabhängig, legt die GUI-Gestaltung nicht fest.
Programmgesteuerte Interaktion: Benutzer reagiert auf Programm.
Ereignisgesteuerte Interaktion: Programm reagiert auf Benutzer.
Fenster: Rechteckiger Bereich des Bildschirms. Es gibt eine Baumstruktur aller Fenster, der Bildschirm ist die Wurzel *(z.B. Dialogbox, Scrollbar, Button…)*.
Display: Rechner mit Tastatur, Zeigegerät und $1..m$ Bildschirme
X Client: Applikation, die einen Display nutzen will. Kann lokal oder entfernt laufen.
X Server: Softwareteil des X Window System, der ein Display ansteuert. Beim Nutzer.

GUI ARCHITEKTUR

Nicht nur X Window System, sondern auch **Window Manager** *(Verwaltung der sichtbaren Fenster, Umrandung, Knöpfe. Läuft im Client und realisiert Window Layout Policy)* und **Desktop Manager** *(Desktop-Hilfsmittel wie Taskleiste, Dateimanager, Papierkorb etc.)*.

XLIB

Ist das **C Interface** für das X Protocol. Wird meist nicht direkt verwendet.
Funktionen: `XOpenDisplay()` *öffnet Verbindung zum Display, NULL = Wert von `DISPLAY` Umgebungsvariabel)*, `XCloseDisplay()` *schliesst Verbindung*, `XCreateSimpleWindow()` *erzeugt ein Fenster*, `XDestroyWindow()` *entfernt ein Fenster & Unterfenster*, `XMapWindow()` *bestimmt, dass ein Fenster angezeigt werden soll (unhide)*, `XMapRaised()` *bringt Fenster in den Vordergrund*, `XMapSubwindows()` *zeigt alle Unterfenster an, Expose Event*, `XUnmapWindow()` *versteckt Fenster*, `XUnmapSubwindows()` *versteckt Unterfenster, `UnmapNotify` Event*

X Protocol: Legt die **Formate für Nachrichten** zwischen X Client und Server fest. **Requests** *(Dienstanforderungen, Client → Server)*, **Replies** *(Antworten auf Requests, Client ← Server)*, **Events** *(Ereignismeldungen, Client ← Server, doppelt gepuffert)*, **Errors** *(Fehlermeldungen auf vorangegangene Requests, Client ← Server)*
Request Buffer: Nachrichtenpufferung auf der Client Seite. Möglichst wenig Anforderungsübertragungen an X Server. **Ziel**: möglichst wenige «Anforderungsübertragungen» an Server. Gruppierung für Kommunikationseffizienz.
Pufferung bei Ereignissen: Werden beim X Server und beim Client gepuffert. **Server Puffer** berücksichtigt Netzwerkverfügbarkeit, **Client Puffer** wartet bis Client Events abholt `XNextEvent()`.
Puffer-Leerung: Client: wartet blockierend auf Event, benötigt Client-Request Reply von Server, `XFlush` **X Event Handling:** Ereignisse werden vom Client verarbeitet oder weitergeleitet. Muss **festlegen, welche** Typen er empfangen will. `XSelectInput()` *legt fest, welche Events via Event-Masken empfangen werden, z.B. `ExposureMask`*, `XNextEvent()` *kopiert den nächsten Event aus dem Buffer*.

ZEICHNEN

Ressourcen: Server-seitige Datenhaltung zur Reduktion des Netzwerkverkehrs. Halten Informationen im Auftrag von Clients. Diese identifizieren Informationen mit IDs. Kein Hin- und Herkopieren komplexer Datenstrukturen nötig. *(z.B. Window, Pixmap, Colormap, Font, Graphics-Context)*
Pufferung verdeckter Fensterinhalte: Minimal *(keine Pufferung)* oder **Optional** *(Hintergrundspeicher zum Sichern vorhanden)*
Pixmap: Server-Seitiger **Grafikspeicher**, wird immer gecached.
X Grafikfunktionen: Bilddarstellung mittels **Rastergrafik** und **Farbtabelle**. Erlauben das Zeichnen von Figuren, Strings und Texten. **Ziele** für das Zeichnen können Fenster oder Pixmap sein.
Graphics Context: Legt diverse **Eigenschaften** fest, die Systemaufrufe nicht direkt unterstützen *(z.B. Liniendicke, Farben, Füllmuster)*. Client kann mehrere GCs gleichzeitig nutzen.

FENSTER SCHLIESSEN

Schaltfläche wird vom **Window Manager** erzeugt. X weiss **nichts** über spezielle Bedeutung der Schaltfläche, der Window Manager schliesst das Fenster. Es gibt ein **Protokoll** zwischen Window Manager und Applikation. `ClientMessage` Event mit `WM_DELETE_MESSAGE`.
Atoms: ID eines Strings, der für **Meta-Zwecke** benötigt wird. Erspart Parsen der Strings.
Properties: Werden mit jedem Fenster assoziiert. **Generischer Kommunikations-Mechanismus** zwischen Applikation und Window Manager.
`WM_PROTOCOLS`: Von X Standard definierte Anzahl an Protokollen, die der Window Manager verstehen soll. Ein Client kann sich für Protokolle **registrieren**.
`WM_DELETE_WINDOW`: Wird beim Drücken des «x» vom Window Manager an den Client geschickt.